

LAB 2: CHE655

NAME: AADITYAAMLAN PANDA

ROLL: 220007

Introduction

The code tries to find the A = QR decomposition of a matrix and validate the results of a given QR decomposition

Methodology

- find_col_norm function calculates norm of a column of a given matrix
- find_vec_norm function calculates norm of a given vector
- find_dot function calculates the product of two column vectors from given column of two given matrices

```
import numpy as np
import math

def find_col_norm(V,pos,n):
    s = 0
    for i in range(n):
        s+=V[i][pos]*V[i][pos]

    s = math.sqrt(s)
    return s

def find_dot(A,Q,apos,qpos,n):
    s = 0
    for i in range(n):
        s+=A[i][apos]*Q[i][qpos]

    return s

def find_vec_norm(V,n):
    s = 0
    for i in range(n):
        s+=V[i]*V[i]

    s = math.sqrt(s)
    return s
```

- Input the matrix A

```
print('Enter the number of rows = number of columns and then enter the elements at every position')
N = int(input('Enter the number of rows = number of columns:'))

A = np.zeros((N,N))
for i in range(N):
    for j in range(N):
        A[i][j] = float(input("Enter element at position "+str(i)+" "+str(j)+" :"))
print (find_col_norm(A,0,N))
print(find_dot(A,A,0,1,N))
```

- Q1 is calculated

```

Q = np.zeros((N,N))
R = np.zeros((N,N))

# Finding q1
norm_a1 = find_col_norm(A,0,N)
for i in range(N):
    Q[i][0] = A[i][0]/norm_a1

print(norm_a1)
print(Q)

```

- Since we may have to calculate the dot products again and again dp matrix helps in referencing to the same dot products while vis matrix helps in visitation dp(i,j) is the dot product of A(i) and Q(j)

```

#defining dp for overlapping subproblems....row stands for A and column stands for Q
dp = np.zeros((N,N))
vis = np.zeros((N,N))

```

- The main algorithm of finding Q is the following. Perp stands for orthogonal projection of A on Q

```

# Finding qr
for j in range(N):
    if j==0:
        continue
    perp = np.zeros(N)
    for i in range(N):
        s = 0

        for r in range(j):

            if vis[j][r] == 0:
                dp[j][r] = find_dot(A,Q,j,r,N)
                vis[j][r] = 1

            s+=dp[j][r]*Q[i][r]

        perp[i] = A[i][j]-s

    perp_norm = find_vec_norm(perp,N)

    for i in range(N):
        Q[i][j] = perp[i]/perp_norm

print(Q)

```

- Finally R is calculated as an upper triangular matrix

```
#Finding R: Rij = Qi*Aj
for i in range(N):
    for j in range(N):
        if i<=j:
            if vis[j][i]==0:
                dp[j][i] = find_dot(A,Q,j,i,N)
                vis[j][i] = 1
            R[i][j] = dp[j][i]
print(R)
```

Validation

Given Results:

$$A = \begin{bmatrix} 3 & 4 & 1 \\ 3 & 5 & 1 \\ 2 & 2 & 1 \end{bmatrix} \quad Q = \begin{bmatrix} 0.64 & -0.2 & -0.74 \\ 0.64 & 0.67 & 0.37 \\ 0.42 & -0.71 & 0.56 \end{bmatrix} \quad R = \begin{bmatrix} 4.7 & 6.6 & 1.7 \\ 0 & 1.15 & -0.24 \\ 0 & 0 & 0.19 \end{bmatrix}$$

Obtained results:

Enter the number of rows = number of columns:3

Enter element at position 0,0:3

Enter element at position 0,1:4

Enter element at position 0,2:1

Enter element at position 1,0:3

Enter element at position 1,1:5

Enter element at position 1,2:1

Enter element at position 2,0:2

Enter element at position 2,1:2

Enter element at position 2,2:1

4.69041575982343

31.0

4.69041575982343

[[0.63960215 0. 0.]

[0.63960215 0. 0.]

[0.42640143 0. 0.]]

[[0.63960215 -0.1979519 -0.74278135]

[0.63960215 0.67303645 0.37139068]

[0.42640143 -0.71262682 0.55708601]]

[[4.69041576 6.60922221 1.70560573]

[0. 1.14812099 -0.23754227]

[0. 0. 0.18569534]]

The last and second last matrices show the values of Q and R respectively: -

Q: -

[[0.63960215 -0.1979519 -0.74278135]

[0.63960215 0.67303645 0.37139068]

[0.42640143 -0.71262682 0.55708601]]

R: -

```
[[ 4.69041576  6.60922221  1.70560573]
 [ 0.         1.14812099 -0.23754227]
 [ 0.         0.         0.18569534]]
```

The obtained results match well with given results

What happens when rank is less than n: -

In that case the columns will become linearly dependent and hence will not span the matrix completely. Thus one of the perp or orthogonal projections would end up getting 0 and will result in norm zero. As norm of perp becomes denominator while obtaining the columns of q via orthonormalisation, it would fail the decomposition step. Trying it out with an input: -

Enter the number of rows = number of columns and then enter the elements at every position

Enter the number of rows = number of columns:2

Enter element at position 0,0:1

Enter element at position 0,1:2

Enter element at position 1,0:2

Enter element at position 1,1:4

2.23606797749979

10.0

2.23606797749979

```
[[0.4472136  0.      ]
```

```
 [0.89442719 0.      ]]
```

```
[[0.4472136      nan]
```

```
 [0.89442719      nan]]
```

```
[[2.23606798 4.47213595]
```

```
 [0.          nan]]
```

c:\users\aadityaap22\.spyder-py3\aaditya_amlan_panda_220007_lab1_code.py:81:

RuntimeWarning: invalid value encountered in scalar divide

```
Q[i][j] = perp[i]/perp_norm
```

As we can see in the above the rank of the matrix is less than 2 and QR decomposition fails.